

A Jacobian-Weighted Volumetric Density Correction for Mass Conservation in Curvilinear Lattice-Boltzmann Flows

Corresponding author
Periodic-Hill D3Q19 GILBM Solver Group

June 26, 2026

Abstract

Body-fitted (curvilinear) lattice-Boltzmann (LBM) simulations of wall-bounded turbulence accumulate a slow, mesh-dependent drift of the global mean density. The drift is not a physical mass source but the residual compressibility error of the weakly-compressible time stepping, and on a stretched grid it cannot be removed by the naive device of summing nodal densities with unit weight: such a sum integrates over *computational* space and weights each physical region by the inverse local Jacobian, biasing the controlled mean toward the node-clustered (small-cell) zones. We formulate the only mesh-invariant remedy: a *volume-weighted* density governor that regulates the genuinely physical mass $M = \int_{\Omega} \rho dV$. Each lattice node is assigned a median-dual control volume w_p , and the volume-weighted mean $\langle \rho \rangle_V = \sum_p w_p \rho_p / \sum_p w_p$ is restored to unity by a single, spatially uniform shift injected into the rest population f_0 . We prove three exact properties of the corrector: the median-dual weights form an exact partition of unity ($\sum_p w_p = V_{\text{mesh}}$), the shift is momentum-silent (the rest velocity is $\mathbf{e}_0 = \mathbf{0}$), and the unit mean is recovered in a single application (unit-gain fixed point). The cell volumes that build w_p are computed two ways: an exact straight-chord Shoelace area (Method 0) and a 3×3 Gauss–Legendre quadrature of the in-plane Jacobian J_{2D} reconstructed by degree-5 Lagrange interpolation (Method 1). A host-only verification harness confirms the Gauss–Legendre rule is exact through degree 5, that Method 1 converges at sixth order on a curved, genuinely two-dimensional (non-separable) test map while the chorded Method 0 is locked at $O(h^2)$, and that the mass governor restores $\langle \rho \rangle_V = 1$ to reduction-rounding precision ($\sim 10^{-12}$ at production scale, $\sim 10^{-16}$ in the small verification harness) while preserving momentum exactly. We give a complete theory $\leftrightarrow$ code map for the D3Q19 GILBM implementation and an independent record verifying that the MRT collision operator and Guo forcing inject no spurious mass, leaving the volumetric corrector as the sole, controlled mass channel.

摘要

貼體（曲線）格子波茲曼（LBM）模擬牆束湍流時，全域平均密度會隨時間緩慢漂移，且此漂移與網格相關。該漂移並非物理質量源，而是弱可壓縮時間推進殘留的可壓縮性誤差；在拉伸網格上，無法以「對節點密度取單位權重求和」這種素樸方式消除：此種求和實為在計算空間上積分，會以局部 Jacobian 的倒數加權每個物理區域，使受控平均偏向節點密集（小網格）區。本文提出唯一與網格無關的修正法：以體積加權的密度調節器，調控真正的物理質量 $M = \int_{\Omega} \rho dV$ 。每個格點賦予中位對偶（median-dual）控制體積 w_p ，並透過注入靜止分布 f_0 的單一均勻位移，將體積加權平均 $\langle \rho \rangle_V = \sum_p w_p \rho_p / \sum_p w_p$ 一次回復為 1。我們證明修正子的三項精確性質：中位對偶權重構成精確的單位分割（ $\sum_p w_p = V_{\text{mesh}}$ ）、位移不影響動量（靜止速度 $\mathbf{e}_0 = \mathbf{0}$ ）、以及單次套用即回復單位平均（單位增益不動點）。建立 w_p 所需的網格體積以兩法計算：精確直弦 Shoelace 面積（方法 0），以及對以五次 Lagrange 內插重建之平面內 Jacobian J_{2D} 施以 3×3 Gauss–Legendre 求積（方法 1）。獨立主機端驗證確認該求積對五次以下多項式精確，方法 1 在一個真正二維（不可分離）的曲線測試映射上達六階收斂、而直弦方法 0 鎖在 $O(h^2)$ ，且質量調節器在保持動量精確守恆下將 $\langle \rho \rangle_V$ 回復至約化捨入精度（生產規模約 $\sim 10^{-12}$ 、小型驗證程式約 $\sim 10^{-16}$ ）。文中並給出 D3Q19 GILBM 實作的完整理論↔程式碼對照，以及驗證 MRT 碰撞算子與 Guo 體積力不注入任何虛假質量的獨立紀錄，使體積修正子成為方案中唯一受控的質量通道。

Keywords: lattice Boltzmann method; mass conservation; curvilinear / body-fitted mesh; median-dual control volume; Gauss–Legendre quadrature; Jacobian metric; partition of unity; periodic hill; MRT collision; Guo forcing.

Contents

1	Introduction	3
2	Governing framework and notation	4
3	Volumetric mass-conservation theory	6
3.1	The physical mass invariant	6
3.2	Quadrature of the physical mass and the mean density	7
3.3	The correction operator: a discrete mass governor	7
3.3.1	Definition	7
3.3.2	(a) Mass change is exactly $\delta\rho$ per node	7
3.3.3	(b) Momentum is exactly invariant	8
3.3.4	(c) One-shot restoration of the unit mean	8
3.3.5	Control-theoretic interpretation	8
3.4	Relation to the lattice-Boltzmann moments and forcing	8
4	Control-volume weights and the partition-of-unity identity	9
4.1	Cell volumes	9
4.2	Median-dual nodal weight	10
4.3	Partition-of-unity proof	10
5	Cell-volume quadrature: Shoelace vs. Jacobian GL-3	11
5.1	N1: two quadratures of the same area integral	11
5.2	N2: degree-5 Lagrange reconstruction at the Gauss points	12

6	Numerical analysis and error budget	13
6.1	N3: floating-point exactness of the correction	13
6.2	N4: why the curved domain prefers Method 1	14
6.3	Error budget	14
7	Numerical experiments	15
7.1	E1: GL-3 polynomial exactness	15
7.2	E2: h -refinement convergence	15
7.3	E3: one-shot mass restoration and momentum preservation	16
7.4	E4: partition-of-unity on the production grid — a bit-exactness audit	17
8	Implementation: mapping the mathematics to source code	19
8.1	Theory $\leftrightarrow$ code correspondence	19
8.2	Key kernels (verbatim)	19
8.2.1	Shoelace cell volume	19
8.2.2	GL-3 data, Lagrange basis, and J_{2D} reconstruction	20
8.2.3	Jacobian-GL nodal weights (core triple loop)	21
8.2.4	Average, correction, and the rest-population apply site	22
8.3	Implementation notes	23
9	Verification of the collision operator	24
10	Discussion and limitations	24
10.1	Discrete self-consistency, not field fidelity, is the correctness criterion	24
11	Conclusion	26
A	MRT collision verification record	27

1 Introduction

The lattice-Boltzmann method (LBM) has matured into a competitive solver for wall-bounded turbulence, particularly in the weakly-compressible, single-relaxation- or multiple-relaxation-time (MRT) form [1, 2, 4, 12]. Its native Cartesian lattice, however, is awkward for the smooth, curved walls that characterise canonical separated flows such as the periodic-hill benchmark [10, 11]. Two classical routes restore geometric flexibility: interpolation-supplemented LBM on body-fitted meshes [5, 6], and finite-volume formulations on unstructured/curvilinear grids [9]. In all of these the populations are advanced on a logical (computational) lattice whose physical image is stretched and curved.

A subtle consequence of this stretching is a slow, mesh-dependent drift of the global mean density. In a weakly-compressible scheme the density is a dynamical field, and the time stepping leaves a residual compressibility error that, on a non-uniform body-fitted grid, accumulates into a global offset of $\langle \rho \rangle$ away from its lattice reference value of unity. Left unchecked, this offset perturbs the pressure (through the equation of state $p = c_s^2 \rho$) and contaminates long-time turbulence statistics.

The naive cure—summing the nodal densities with *unit* weight and shifting the field to drive that sum to a target—is *wrong* on a stretched mesh. As we show in §3.1, a unit-weight sum is a quadrature of the density over *computational* space, and therefore weights each physical sub-region by the *inverse* local Jacobian. Where grid lines cluster (near the wall and in the lee of the hill)

the metric is small and the unit-weight sum over-counts; where cells are coarse it under-counts. Regulating such a biased surrogate drives the density toward a spatially skewed mean and injects a spurious, mesh-dependent body force into the momentum equation.

The correct invariant to monitor is the genuinely physical mass $M = \int_{\Omega} \rho dV$, whose discrete form requires each node to carry a *physical control volume* w_p . This paper develops, analyses, verifies, and documents the implementation of a *Jacobian-weighted volumetric density corrector* built on this principle. Its contributions are:

- a continuous/discrete theory (§3) showing that only the volume-weighted mean $\langle \rho \rangle_V$ is a mesh-invariant target, and proving that a uniform shift into the rest population f_0 restores $\langle \rho \rangle_V = 1$ in one step while preserving momentum exactly;
- a median-dual control-volume construction with an exact partition-of-unity identity (§4);
- two quadratures of the per-cell area—an exact straight-chord Shoelace area and a 3×3 Gauss–Legendre integration of the in-plane Jacobian reconstructed by degree-5 Lagrange interpolation—together with a full error budget (§5–§6) [7, 8];
- an independent host-only verification harness confirming GL-3 exactness, sixth-order convergence of the Jacobian method, and algebraically exact, momentum-preserving mass restoration (§7);
- a faithful theory $\leftrightarrow$ code map for the D3Q19 GILBM solver (§8) and a verification record for the MRT collision operator and Guo forcing [3] establishing that the corrector is the sole controlled mass channel (§9, Appendix A).

2 Governing framework and notation

Lattice and moments. The solver advances a D3Q19 lattice-Boltzmann scheme with $N_Q = 19$ discrete velocities $\mathbf{e}_q$ ($q = 0, \dots, 18$), rest velocity $\mathbf{e}_0 = (0, 0, 0)$, and lattice weights W_q with $W_0 = \frac{1}{3}$. The hydrodynamic moments are the velocity-space sums

$$\rho = \sum_{q=0}^{18} f_q, \quad \rho \mathbf{u} = \sum_{q=0}^{18} \mathbf{e}_q f_q. \quad (1)$$

Collision is performed by an MRT operator on the moment basis M with relaxation matrix S , projected to velocity space as $K = M^{-1}SM$; the body force is applied by Guo forcing [3] at a compile-time Hermite order `FORCE_HERMITE_ORDER`. The conserved (density and momentum) moments are relaxed at rate zero; the verification of this property is recorded in §9 and Appendix A.

Curvilinear map (unified convention). Throughout the paper we use a single coordinate convention. Let (ξ, η, ζ) be the logical (computational) coordinates, identified with the integer lattice indices (j, i, k) as

logical coordinate	lattice index	role
ξ	j	streamwise
η	i	spanwise (homogeneous)
ζ	k	wall-normal (carries the hill)

The body-fitted mesh is an *extrusion*: the spanwise physical coordinate depends on the spanwise index alone, $x = x(\eta)$, with straight grid lines and (non-uniform) spacing $\Delta x_i = x_{i+1} - x_i$, while the cross-section $(y, z) = (y(\xi, \zeta), z(\xi, \zeta))$ is a genuinely two-dimensional curvilinear plane, z being the wall-normal physical coordinate. The map is

$$\Phi : (\xi, \eta, \zeta) \mapsto (x(\eta), y(\xi, \zeta), z(\xi, \zeta)). \quad (2)$$

Because x depends on the spanwise variable only and (y, z) on the in-plane variables only, the 3×3 Jacobian is block-diagonal and its determinant factors:

$$\mathcal{J}_3 = \det \frac{\partial(x, y, z)}{\partial(\xi, \eta, \zeta)} = \pm x'(\eta) J_{2D}, \quad J_{2D} \equiv \frac{\partial(y, z)}{\partial(\xi, \zeta)} = y_\xi z_\zeta - y_\zeta z_\xi, \quad (3)$$

where subscripts denote partial derivatives and J_{2D} is the in-plane (cross-section) Jacobian computed and stored by the solver (code symbol `J_2D`). The overall sign of $\mathcal{J}_3$ is orientation-dependent (here $x'(\eta)$ occupies the second column, contributing a cofactor sign $(-1)^{1+2}$); it is immaterial in practice because the code builds the volume element from the absolute value (`fabs`). The physical volume element is therefore

$$dV = |\mathcal{J}_3| d\xi d\eta d\zeta = |x'(\eta)| J_{2D} d\xi d\eta d\zeta, \quad (4)$$

i.e. a physical cell volume is the spanwise chord length $|\Delta x_i|$ times an area measured in the (y, z) cross-section. Nodes are indexed $p = (i, j, k)$ and mesh cells $c = (i_c, j_c, k_c)$; h denotes a representative physical cell size, $n \sim 10^6$ the number of interior reduction nodes, and $\varepsilon = 2.2 \times 10^{-16}$ the double-precision unit roundoff.

The next lemma justifies why the in-plane factor J_{2D} of (3) is the correct *area density* of the cross-section: it is precisely the square root of the determinant of the first fundamental form, so the determinant—not the product of the per-direction scale factors—is the quantity that the cell-volume quadratures of §5 must integrate.

Lemma 1 (Metric form of the in-plane area element). *Let the in-plane map $(\xi, \zeta) \mapsto (y(\xi, \zeta), z(\xi, \zeta))$ of (2) have covariant basis vectors $\mathbf{r}_\xi = (y_\xi, z_\xi)$ and $\mathbf{r}_\zeta = (y_\zeta, z_\zeta)$, with scale factors (Lamé coefficients)*

$$h_\xi = |\mathbf{r}_\xi| = \sqrt{y_\xi^2 + z_\xi^2}, \quad h_\zeta = |\mathbf{r}_\zeta| = \sqrt{y_\zeta^2 + z_\zeta^2}, \quad (5)$$

and first-fundamental-form (metric) components $g_{\xi\xi} = h_\xi^2$, $g_{\zeta\zeta} = h_\zeta^2$, and $g_{\xi\zeta} = \mathbf{r}_\xi \cdot \mathbf{r}_\zeta = y_\xi y_\zeta + z_\xi z_\zeta$. Then the physical area element of the cross-section is

$$dA = \sqrt{\det g} d\xi d\zeta, \quad \det g = g_{\xi\xi} g_{\zeta\zeta} - g_{\xi\zeta}^2, \quad (6)$$

and its area density equals the in-plane Jacobian of (3) in absolute value,

$$\sqrt{\det g} = |y_\xi z_\zeta - y_\zeta z_\xi| = |J_{2D}| = h_\xi h_\zeta \sin \varphi, \quad (7)$$

where φ is the angle between the coordinate-line tangents $\mathbf{r}_\xi$ and $\mathbf{r}_\zeta$. In particular $|J_{2D}| = h_\xi h_\zeta$ (the scale-factor product) holds if and only if the coordinate lines are orthogonal, i.e. $g_{\xi\zeta} = 0$, $\varphi = \pi/2$.

Proof. Expand the determinant of the metric directly,

$$\det g = (y_\xi^2 + z_\xi^2)(y_\zeta^2 + z_\zeta^2) - (y_\xi y_\zeta + z_\xi z_\zeta)^2. \quad (8)$$

Expanding, $(y_\xi^2 + z_\xi^2)(y_\zeta^2 + z_\zeta^2) = y_\xi^2 y_\zeta^2 + y_\xi^2 z_\zeta^2 + z_\xi^2 y_\zeta^2 + z_\xi^2 z_\zeta^2$ and $(y_\xi y_\zeta + z_\xi z_\zeta)^2 = y_\xi^2 y_\zeta^2 + 2 y_\xi y_\zeta z_\xi z_\zeta + z_\xi^2 z_\zeta^2$. The pure terms $y_\xi^2 y_\zeta^2$ and $z_\xi^2 z_\zeta^2$ cancel between the two, leaving

$$\det g = y_\xi^2 z_\zeta^2 + z_\xi^2 y_\zeta^2 - 2 y_\xi z_\zeta y_\zeta z_\xi = (y_\xi z_\zeta - y_\zeta z_\xi)^2 = J_{2D}^2, \quad (9)$$

which is the two-dimensional Lagrange identity $|\mathbf{a}|^2 |\mathbf{b}|^2 - (\mathbf{a} \cdot \mathbf{b})^2 = (\mathbf{a} \times \mathbf{b})^2$ for $\mathbf{a} = \mathbf{r}_\xi$, $\mathbf{b} = \mathbf{r}_\zeta$. Taking the positive square root gives $\sqrt{\det g} = |J_{2D}|$, and the standard surface-area element $dA = \sqrt{\det g} d\xi d\zeta$ then yields (6)–(7). Finally, writing $\cos \varphi = g_{\xi\zeta}/(h_\xi h_\zeta)$,

$$\det g = h_\xi^2 h_\zeta^2 (1 - \cos^2 \varphi) = (h_\xi h_\zeta \sin \varphi)^2, \quad (10)$$

so $|J_{2D}| = h_\xi h_\zeta \sin \varphi$, with equality to $h_\xi h_\zeta$ exactly when $\sin \varphi = 1$, i.e. $g_{\xi\zeta} = 0$. $\square$

Remark 1 (Why the scale-factor product over-counts a skewed grid). The naive product $h_\xi h_\zeta d\xi d\zeta$ is the area of the *rectangle* with sides $h_\xi d\xi$ and $h_\zeta d\zeta$; the actual cell is the *parallelogram* spanned by $\mathbf{r}_\xi d\xi$ and $\mathbf{r}_\zeta d\zeta$, whose area is $h_\xi h_\zeta \sin \varphi = |J_{2D}|$ by (7). The two agree only on an orthogonal grid; on a skewed grid ($g_{\xi\zeta} \neq 0$) the scale-factor product over-counts the area by the factor $1/\sin \varphi$. The Steger–Sorenson elliptic (Poisson) generator promotes but does not enforce orthogonality, so in general $g_{\xi\zeta} \neq 0$ (the grid is only near-orthogonal); wherever $g_{\xi\zeta} \neq 0$ the determinant J_{2D} , and not the product $h_\xi h_\zeta$, is the correct area density. The discrete Shoelace area (Method 0) likewise returns the true straight-chord parallelogram/quadrilateral area and so also accounts for skew; both cell-volume estimators of §5 therefore integrate the metric area element $|J_{2D}|$ of Lemma 1, never the scale-factor product.

3 Volumetric mass-conservation theory

3.1 The physical mass invariant

The conserved invariant of the incompressible/weakly-compressible flow is the *physical* mass

$$M = \int_{\Omega} \rho dV = \int_{\Omega_c} \rho |\mathcal{J}_3| d\xi d\eta d\zeta, \quad (11)$$

where $\Omega \subset \mathbb{R}^3$ is the physical fluid domain and Ω_c its logical pre-image, a uniform cuboid in (ξ, η, ζ) .

A naive “mass” formed by summing the nodal density over computational indices with *unit* weight,

$$\widetilde{M} = \sum_p \rho_p \approx \int_{\Omega_c} \rho d\xi d\eta d\zeta = \int_{\Omega} \frac{\rho}{|\mathcal{J}_3|} dV, \quad (12)$$

is *not* a discretization of (11): it integrates over computational space and therefore weights each physical sub-region by the inverse local Jacobian $|\mathcal{J}_3|^{-1}$. On a stretched body-fitted mesh the metric varies strongly—grid lines cluster near the wall and in the lee of the hill where $|\mathcal{J}_3|$ is small—so $\widetilde{M}$ over-counts the node-clustered (small-cell) regions and under-counts the coarse-cell regions. Controlling $\widetilde{M}$ would drive the density field toward a spatially *biased* mean and inject a spurious, mesh-dependent body force into the momentum equation. Only the genuinely physical functional (11), weighted by $|\mathcal{J}_3|$, is the correct invariant to monitor and to regulate.

3.2 Quadrature of the physical mass and the mean density

Discretize (11) by a nodal quadrature rule. Let $p = (i, j, k)$ index the unique interior lattice nodes and let $w_p > 0$ be the *physical control volume* assigned to node p (constructed in §4). The mass and total volume are approximated by

$$M \approx \sum_p \rho_p w_p, \quad (13)$$

$$V = \sum_p w_p \approx \int_{\Omega} dV = |\Omega|, \quad (14)$$

and the *volume-weighted mean density* is their ratio,

$$\langle \rho \rangle_V \equiv \frac{M}{V} = \frac{\sum_p \rho_p w_p}{\sum_p w_p}. \quad (15)$$

By construction $\langle \rho \rangle_V$ is the exact discrete average that equals unity when $\rho_p \equiv 1$ for all p , *independently of the mesh stretching*, because the same weights w_p appear in numerator and denominator. This is the defining property the unit-weight surrogate (12) lacks. In the implementation the weighted sum $\sum_p \rho_p w_p$ is evaluated by a GPU reduction over the unique interior node set $i \in [3, N_x-5]$, $j \in [3, N_{y,d}-5]$, $k \in [3, N_z-4]$, summed across MPI ranks, and divided by the pre-computed global volume $V = \sum_p w_p$ to yield $\langle \rho \rangle_V$ on the root rank. The fidelity of (15) as a model of (11) rests entirely on how faithfully w_p reproduces the physical control volume, constructed in §4.

3.3 The correction operator: a discrete mass governor

3.3.1 Definition

At every time step the solver forms $\langle \rho \rangle_V$ from (15) and defines the scalar correction

$$\delta \rho \equiv 1 - \langle \rho \rangle_V, \quad (16)$$

the target reference density being unity in lattice units. The root rank computes $\delta \rho$ and broadcasts it to all ranks. It is then applied as a *uniform* additive shift, injected entirely into the rest population f_0 at every interior node p :

$$\rho_p \leftarrow \rho_p + \delta \rho, \quad f_{0,p} \leftarrow f_{0,p} + \delta \rho. \quad (17)$$

We now establish three exact properties.

3.3.2 (a) Mass change is exactly $\delta \rho$ per node

Since $\rho_p = \sum_q f_{q,p}$ and only f_0 is altered,

$$\Delta \rho_p = \sum_q \Delta f_{q,p} = \Delta f_{0,p} = \delta \rho. \quad (18)$$

The increment is identical at every node, so the change in the discrete volume-weighted mass is

$$\Delta M = \sum_p w_p \Delta \rho_p = \delta \rho \sum_p w_p = \delta \rho V. \quad (19)$$

3.3.3 (b) Momentum is exactly invariant

The D3Q19 rest velocity is $\mathbf{e}_0 = (0, 0, 0)$. The momentum moment changes by

$$\Delta(\rho u_\alpha)_p = \sum_q e_{q,\alpha} \Delta f_{q,p} = e_{0,\alpha} \delta\rho = 0, \quad \alpha \in \{x, y, z\}. \quad (20)$$

Because the carrier population has zero lattice velocity, the injected mass carries *no* momentum: the correction lives in the kernel of every momentum moment. This is precisely why the rest population f_0 , and not any moving population, is chosen as the carrier—any $\mathbf{e}_q \neq \mathbf{0}$ would couple the mass governor to the momentum equation and perturb the velocity field.

3.3.4 (c) One-shot restoration of the unit mean

Apply (17) and recompute the mean using (15) and the constancy of $\delta\rho$,

$$\langle\rho\rangle_V^{\text{new}} = \frac{\sum_p w_p (\rho_p + \delta\rho)}{\sum_p w_p} = \frac{\sum_p w_p \rho_p}{\sum_p w_p} + \delta\rho = \langle\rho\rangle_V + (1 - \langle\rho\rangle_V) = 1. \quad (21)$$

A spatially uniform shift moves the volume-weighted mean by exactly its own magnitude, so the target $\langle\rho\rangle_V = 1$ is reached in a *single* application; no iteration is required. The only residual is the floating-point rounding of the parallel reduction that forms $\langle\rho\rangle_V$.

3.3.5 Control-theoretic interpretation

Equations (16)–(21) constitute a discrete proportional “mass governor” with *unit feedback gain*. Writing $\chi^{(n)} = \langle\rho\rangle_V$ at step n before correction, the corrector realizes the fixed-point map

$$\chi \mapsto \chi + (1 - \chi), \quad (22)$$

whose unique fixed point $\chi^* = 1$ is reached in one step with zero contraction remainder (the error multiplier is $1 - \text{gain} = 0$). The scheme is unconditionally stable in this idealized sense: it neither under- nor over-shoots, and introduces no oscillatory feedback mode. In the running solver the correction is applied with a one-step lag— $\langle\rho\rangle_V$ is measured at the end of step n and the shift is applied at step $n+1$ —so the governor removes the compressibility drift accumulated over a single streaming/collision update each step, holding the global mean density pinned at unity to reduction-rounding precision throughout the simulation.

3.4 Relation to the lattice-Boltzmann moments and forcing

The corrector is consistent with the moment structure of the LBM. A perturbation $\Delta f_q = \delta\rho \delta_{q0}$ has moment signature

$$\sum_q \Delta f_q = \delta\rho, \quad \sum_q \mathbf{e}_q \Delta f_q = \mathbf{e}_0 \delta\rho = \mathbf{0}, \quad (23)$$

so the injection populates the density moment alone and lies in the common kernel of all three momentum moments: it is a pure, momentum-silent density shift. This is the moment-space statement of (20).

The mass governor is complementary to the body force. The solver applies Guo forcing [3] with Hermite expansion order controlled at compile time. At first order the forcing population is

$$F_q = W_q (3 c_{q,y}) \mathcal{F}, \quad (24)$$

(pure lattice-velocity weighting, with $\mathcal{F}$ the driving force and $c_{q,y} = e_{q,y}$), whose density moment vanishes identically,

$$\sum_q F_q = 3\mathcal{F} \sum_q W_q c_{q,y} = 0 \iff \sum_q W_q e_{q,y} = 0, \quad (25)$$

the identity holding by the lattice isotropy of the weights. Hence the first-order body force injects *no* net mass; the full second-order Guo form, $F_q = W_q \mathcal{F}[3(c_{q,y} - u_y) + 9(\mathbf{c}_q \cdot \mathbf{u})c_{q,y}]$, likewise carries zero density moment by the same weight identities. The accumulated unit-mean drift that the governor removes is therefore *not* forcing-induced mass; it is the residual compressibility error of the weakly-compressible LBM time stepping on the stretched curvilinear mesh. The volume-weighted corrector (16)–(17) is the discrete device that removes exactly this drift, in one momentum-preserving step per iteration, against the physically correct invariant $M = \int_{\Omega} \rho dV$ of §3.1.

4 Control-volume weights and the partition-of-unity identity

4.1 Cell volumes

The mesh cell $c = (i_c, j_c, k_c)$ is the hexahedron spanned by logical indices $[i_c, i_c+1] \times [j_c, j_c+1] \times [k_c, k_c+1]$. By the block structure (3), its physical volume separates into the spanwise chord times a cross-sectional area,

$$V_c = |\Delta x_{i_c}| A_c, \quad \Delta x_{i_c} = x_{i_c+1} - x_{i_c}. \quad (26)$$

Two equivalent constructions of A_c are provided; their detailed numerical analysis is deferred to §5–§6.

Method 0 (Shoelace / surveyor area). The four cross-section corners (y_a, z_a) , $a = 0, \dots, 3$, taken cyclically at logical positions (j_c, k_c) , (j_c+1, k_c) , (j_c+1, k_c+1) , (j_c, k_c+1) , define a planar quadrilateral whose exact area is the Gauss/shoelace formula

$$A_c^{\text{Shoe}} = \frac{1}{2} \left| \sum_{a=0}^3 (y_a z_{a+1} - z_a y_{a+1}) \right|, \quad (a+1) \bmod 4. \quad (27)$$

This is the *exact* area of the straight-chord quadrilateral. When the V_c of adjacent cells are summed, the line integrals along shared edges cancel pairwise (opposite orientation), so the total telescopes to the exact area of the piecewise-linear mesh polygon. Hence Method 0 reproduces the discrete (piecewise-linear) mesh volume *exactly*; its only error is the $O(h^2)$ geometric chording of the truly curved boundary.

Method 1 (Jacobian, 3×3 Gauss–Legendre). Alternatively the area is the metric integral

$$A_c^{\text{Jac}} = \int_{j_c}^{j_c+1} \int_{k_c}^{k_c+1} J_{2D}(\xi, \zeta) d\xi d\zeta \approx \sum_{a=1}^3 \sum_{b=1}^3 W_a W_b J_{2D}(j_c + \xi_a, k_c + \zeta_b), \quad (28)$$

a tensor-product 3-point Gauss–Legendre (GL-3) rule. On the reference cell $[0, 1]$ the nodes and weights are

$$\{\xi_a\} = \left\{ \frac{1-\sqrt{3/5}}{2}, \frac{1}{2}, \frac{1+\sqrt{3/5}}{2} \right\}, \quad \{W_a\} = \left\{ \frac{5}{18}, \frac{8}{18}, \frac{5}{18} \right\}. \quad (29)$$

GL- n integrates polynomials of degree $\leq 2n-1$ exactly; for $n = 3$ the rule is exact through degree 5 per direction [7, 8]. Because the nodes (29) are off-grid, J_{2D} at a Gauss point is reconstructed by a 6-point (degree-5) tensor Lagrange interpolation of nodal J_{2D} values,

$$J_{2D}(\xi, \zeta) \approx \sum_{m=0}^5 \sum_{n=0}^5 \ell_m(\xi) \ell_n(\zeta) J_{2D}(s_j+m, s_k+n), \quad \ell_m(\xi) = \prod_{\substack{r=0 \\ r \neq m}}^5 \frac{\xi - (s+r)}{(s+m) - (s+r)}, \quad (30)$$

with a centered, wall-clamped stencil start s . The reconstruction is $O(h^6)$ in J_{2D} , exactly matching the degree-5 exactness of GL-3, so the interpolation does *not* lower the quadrature order. If the 6-wide stencil is unavailable (domain edge) or the reconstructed J_{2D} is non-finite or ≤ 0 , the cell falls back to the Shoelace value (27). Either way the nodal cell volume is $V_c = |\Delta x_{i_c}| A_c$, and a verification routine (`VerifyPhysicalDomainVolume3D`) checks the assembled weight-sum against the discrete mesh volume: Method 0 reproduces it to relative error below 10^{-12} (printing `PASS`), whereas Method 1 *deliberately* departs by the $O(h^2)$ boundary-curvature term and the routine prints a `WARNING`. The routine only prints `PASS/WARNING`; it does not assert or abort.

4.2 Median-dual nodal weight

The control volume of node p is the median-dual (vertex-centered finite-volume) cell [9]: each of the (at most eight) mesh cells touching p contributes one eighth of its volume,

$$w_p = \frac{1}{8} \sum_{c \ni p} V_c, \quad (31)$$

the factor $1/8$ being the fraction of a hexahedral cell nearest one of its eight corners. Two boundary adaptations are made without altering the weight's meaning: (i) in the periodic spanwise direction the wrap is closed by mapping the first interior layer's left cell $i = 3$ to the cell index $N_x - 5$, so the two physical cells adjacent to the periodic seam are both counted exactly once; (ii) at a solid wall in k a node has cells on one side only, so the inner sum runs over the available (one) cell rather than two.

4.3 Partition-of-unity proof

Proposition 1 (Exact volume closure). *With w_p defined by (31), the nodal weights sum to the total mesh volume,*

$$\sum_p w_p = \sum_c V_c = V_{\text{mesh}}. \quad (32)$$

Proof. Substitute (31) and exchange the order of summation,

$$\sum_p w_p = \frac{1}{8} \sum_p \sum_{c \ni p} V_c = \frac{1}{8} \sum_c V_c |\{p : p \text{ is a corner of } c\}|. \quad (33)$$

Every interior hexahedral cell c has exactly eight corner nodes, so the incidence count is $|\{p : p \in c\}| = 8$ and the prefactor $1/8$ cancels it identically, giving $\sum_p w_p = \sum_c V_c$. The pairing of the $\frac{1}{8}$ shares with the 8 corners is a telescoping over the cell-node incidence relation and is therefore exact in floating point up to the order of summation; it does not depend on cell shape, stretching, or curvature. This closure is an exact-arithmetic identity for a periodic mesh whose seam is *bit-closed*;

the production streamwise grid seam is in fact open by $\sim 3 \times 10^{-6}$, which adds a small $\sim 10^{-7}$ geometric residual to (32), measured and decomposed in §7.4.

The boundary adaptations preserve the count. For the periodic spanwise seam, remapping $i = 3 \rightarrow N_x - 5$ relabels which cell supplies a corner share but leaves each physical cell with exactly its eight one-eighth contributions, so no cell is double-counted or dropped and the incidence count stays 8. At a solid wall, a boundary node legitimately borders fewer cells; the missing neighbours lie outside Ω and contribute no volume, so the telescoping still assigns each *existing* cell's eighth-shares to its existing corner nodes. In every case each cell contributes its full V_c exactly once, proving (32). $\square$

Consequently the denominator in (15), $V = \sum_p w_p$ (assembled by an MPI all-reduce into the global control volume), equals the discrete mesh volume in exact arithmetic (to rounding, and exactly so only on a bit-closed grid; see the refinement in §10.1), and $\langle \rho \rangle_V$ is a true volume average.

5 Cell-volume quadrature: Shoelace vs. Jacobian GL-3

Both cell-volume estimators are quadratures of the *same* area integral

$$A(j, k) = \int_j^{j+1} \int_k^{k+1} J_{2D}(\xi, \zeta) \, d\xi \, d\zeta, \quad V_{\text{cell}}(i, j, k) = |\Delta x_i| A(j, k), \quad (34)$$

and differ only in the order to which they resolve the curvature of J_{2D} .

5.1 N1: two quadratures of the same area integral

Method 0 — Shoelace (exact straight-chord area). The first estimator (`MassCorrectionCellVolume`) replaces the curved cell boundary by its four straight chords and computes the exact area of the resulting planar quadrilateral with the surveyor (Gauss/shoelace) formula (27). Equation (27) is the exact integral of J_{2D} for a *bilinear* map whose edges are straight, i.e. for the piecewise-linear (“discrete”) mesh. When adjacent quadrilaterals are summed, every interior edge contributes with opposite sign in the two cells that share it and cancels; the sum therefore telescopes to the exact area of the discrete-mesh polygon. As a consequence Method 0 reproduces the discrete-mesh volume to machine precision *on a bit-closed periodic mesh* (this is precisely what `VerifyPhysicalDomainVolume3D` checks; see §6.2); on the production grid the streamwise periodic seam is not bit-closed and contributes a $\sim 10^{-7}$ geometric residual that exceeds the verifier’s 10^{-12} pass gate, quantified in §7.4. However, the chords are an $O(h^2)$ approximation of the true curved boundary, so Method 0 carries a fixed *geometric* discretisation error

$$A^{\text{Shoe}}(j, k) - A(j, k) = O(h^2) \cdot \kappa, \quad (35)$$

where κ measures the local boundary curvature. This error does not vanish as the quadrature is refined per cell — it is a property of the chorded domain, not of the integration rule.

Method 1 — Jacobian 3×3 Gauss–Legendre. The second estimator (`ComputeJacobianMassCorrectionWeight`) integrates the in-plane metric J_{2D} with the tensor-product GL-3 rule (28)–(29). The decisive point for the error analysis is that the solver does *not* feed a closed-form J_{2D} to the quadrature: the Gauss abscissae are off-grid, so at every Gauss point J_{2D} is first *reconstructed* from nodal samples by the 6-point (degree-5) tensor Lagrange interpolation of §5.2, and GL-3 then integrates *that* reconstruction. A 1D n -point Gauss rule is exact for polynomials of degree $\leq 2n - 1$; with $n = 3$

this is degree 5 per direction, and the reconstruction is itself a degree-5 polynomial per direction. *GL-3 therefore integrates the degree-5 reconstruction exactly*: the Gauss remainder is identically zero on the quantity actually integrated, so the quadrature contributes *no* error to leading order. The convergence-limiting error of Method 1 is consequently the degree-5 *Lagrange reconstruction* error of the sampled metric,

$$A^{\text{Jac}}(j, k) - A(j, k) = \int_j^{j+1} \int_k^{k+1} (\tilde{J}_{2D} - J_{2D}) d\xi d\zeta = O(h^6) J_{2D}^{(6)}, \quad (36)$$

i.e. the per-cell interpolation defect (39) integrated over the cell. Summed over the $O(h^{-2})$ cells of the cross-section the global area error of Method 1 is $O(h^6)$ —interpolation-limited, not quadrature-limited—versus the $O(h^2)$ geometric chord bias of Method 0. This sixth-order rate is confirmed empirically in Section 7 (the measured orders are ≈ 6.0 down to the round-off floor).

Scope of the Gauss remainder. For completeness we record the classical 1D Gauss remainder

$$R_n = \frac{(b-a)^{2n+1} (n!)^4}{(2n+1) ((2n)!)^3} f^{(2n)}(\theta), \quad \theta \in (a, b), \quad (37)$$

which for $n = 3$ on the computational cell $b - a = 1$ gives the closed-form constant

$$R_3 = \frac{(3!)^4}{7 (6!)^3} J_{2D}^{(6)}(\theta) = \frac{1296}{7 \cdot 373,248,000} J_{2D}^{(6)}(\theta) = \frac{1296}{2,612,736,000} J_{2D}^{(6)}(\theta) \approx 4.96 \times 10^{-7} J_{2D}^{(6)}(\theta). \quad (38)$$

Equation (38) bounds the quadrature-*only* error that would arise if a general smooth J_{2D} with degree- > 5 content were fed *directly* to GL-3. In the solver it is *not* the operative error: the integrand is the degree-5 reconstruction, on which $R_3 \equiv 0$. It is retained only as the bound on the (here null) quadrature contribution; the binding term is the interpolation error (36). The implementation falls back to (27) for any cell where the 6-wide interpolation stencil is unavailable or where the reconstructed J_{2D} is non-finite or ≤ 0 , guaranteeing a strictly positive, finite weight everywhere.

5.2 N2: degree-5 Lagrange reconstruction at the Gauss points

The Gauss abscissae $j + \xi_a$, $k + \zeta_b$ are non-nodal, so J_{2D} must be reconstructed there from the nodal metric. The code (`InterpolateJ2DLagrange6`) uses the tensor product (30). The interpolation error of a degree-5 reconstruction is

$$J_{2D} - \tilde{J}_{2D} = \frac{J_{2D}^{(6)}(\theta)}{6!} \prod_{r=0}^5 (\xi - (s + r)) = O(h^6) J_{2D}^{(6)}. \quad (39)$$

This interpolation defect is the *binding* (dominant) error term of Method 1. GL-3 has degree of exactness 5 and the reconstruction (30) is a degree-5 polynomial, so the quadrature (28) integrates the reconstruction *exactly* and adds nothing to leading order—the Gauss remainder (38) is null on the reconstruction. What survives is precisely the gap between the reconstruction and the true metric, $\tilde{J}_{2D} - J_{2D} = O(h^6)$ of (39); integrated over each cell and summed it sets the global $O(h^6)$ rate (36). Method 1 thus integrates a degree-5 polynomial reconstruction of the smooth metric, capturing the sub-cell curvature that the straight chord of Method 0 ignores, and its accuracy is limited not by the quadrature but by how well that degree-5 reconstruction approximates J_{2D} , i.e. by $J_{2D}^{(6)} h^6$.

Stencil placement. `SelectStencilStart` centres the 6-point window on the target cell by choosing $s = \text{cell} - 2$ and clamping to $[\text{lo}, \text{hi} - 5]$, so that interior cells receive a symmetric stencil while near-boundary cells degrade gracefully to a one-sided stencil rather than reaching past the valid data. In the wall-normal direction the ζ -stencils are restricted to physical nodes ($k = 3, \dots, N_z - 4$): the buffer rows $k = 2$ and $k = N_z - 3$ hold metric values that are not part of the physical domain, so near-wall GL cells use the one-sided physical stencils $k = 3..8$ and $k = N_z - 9..N_z - 4$. Because an interior cell’s centred stencil reaches two rows into the neighbouring MPI subdomain in the streamwise (ξ) direction, the ghost rows of J_{2D} ($j = 0, 1, 2$ and $j = N_y - 3, \dots, N_y - 1$) must be exchanged across ranks before the quadrature loop. The solver performs exactly this halo exchange of J_{2D} prior to invoking Method 1; without it, partition-boundary cells would silently fall back to the lower-order shoelace estimate.

6 Numerical analysis and error budget

6.1 N3: floating-point exactness of the correction

Nodal control volumes and partition of unity. Each interior node p is assigned the median-dual weight $w_p = \frac{1}{8} \sum_{c \in p} V_{\text{cell}}(c)$ of (31), with the periodic- x wrap and one-sided near-wall stencils handled explicitly. Since each cell is shared by its eight corner nodes, the weights form a partition of unity that is exact in floating point up to summation order (Proposition 1),

$$\sum_p w_p = \sum_c V_{\text{cell}}(c) = V, \quad (40)$$

where V is obtained once at start-up by `MPI_Allreduce`.

One-shot algebraic exactness. At every step the solver forms $\langle \rho \rangle_V = \frac{1}{V} \sum_p w_p \rho_p$ and $\delta \rho = 1 - \langle \rho \rangle_V$, then applies the uniform shift $\rho \leftarrow \rho + \delta \rho$ together with $f_0 \leftarrow f_0 + \delta \rho$ at every interior node. Because the lattice rest velocity is $\mathbf{e}_0 = \mathbf{0}$, the shift injects mass but *no* momentum ((20)). Applying the uniform shift and using (40) gives

$$\langle \rho \rangle_V^{\text{new}} = \frac{1}{V} \sum_p w_p (\rho_p + \delta \rho) = \langle \rho \rangle_V + \delta \rho \frac{\sum_p w_p}{V} = \langle \rho \rangle_V + \delta \rho = 1, \quad (41)$$

so the mean density is restored to unity in a *single* application; the only residual is the floating-point rounding of the reduction.

Conditioning of the reduction. The reduced quantity $S = \sum_p w_p \rho_p$ is a sum of strictly positive terms ($w_p > 0$, $\rho_p > 0$), so there is no catastrophic cancellation: the condition number $(\sum_p |w_p \rho_p|) / |\sum_p w_p \rho_p| = 1$. A naive sequential accumulation has a worst-case relative error $O(n\varepsilon)$. The device reduction is, however, only *two-level*: the kernel (`ReduceRhoWeightedSum_Kernel`) performs a balanced binary-tree reduction *within* each $B=256$ -thread block, after which the $\sim n/B$ block partials are summed *sequentially* on the host (`ComputeVolumeWeightedRhoAverageRoot: for(b) rho_LocalWeightedMass += rho_partial_h[b]`), followed by a cross-rank `MPI_Reduce`. The intra-block tree therefore lowers the rounding floor only to $O(\varepsilon n/B)$ with $B=256$ —*not* to the full $O(\varepsilon \log_2 n)$ of a single balanced tree. For $n \sim 10^6$ nodes in double precision,

$$n\varepsilon \approx 2.2 \times 10^{-10}, \quad \frac{n}{B} \varepsilon \approx \frac{10^6}{256} \varepsilon \approx 9 \times 10^{-13}, \quad (42)$$

i.e. the intra-block tree buys roughly a factor $B = 256$ over the naive sequential sum (a realistic floor $\sim 10^{-12}$ at production scale), well short of the $\sim 4 \times 10^{-15}$ that a single fully balanced tree would reach. The remaining caveat is determinism: floating-point addition is non-associative, so the across-rank `MPI_Reduce` is bit-reproducible only for a fixed rank order and fixed local reduction tree. The implementation preserves both, so a given run is reproducible, while a change in rank count perturbs $\delta\rho$ at the $O(\varepsilon n/B)$ level seen in (42).

Mass governor interpretation. With `FORCE_HERMITE_ORDER=2` (full Guo forcing) or its first-order variant, the body force carries zero density moment ((25)), so it injects no net mass; the residual compressibility drift accumulated by the LBM time stepping is what the corrector removes. Equation (41) is therefore a discrete mass governor with unit feedback gain and a one-step lag, whose closed-loop fixed point is $\langle\rho\rangle_V = 1$.

6.2 N4: why the curved domain prefers Method 1

It is tempting to call Method 0 “exact” and stop there, but the two estimators are exact for *different* domains. Method 0 is exact for the chorded, piecewise-linear mesh; Method 1 converges to the true curved physical domain as the metric is resolved. For a periodic hill whose lower wall is a smooth curve, the chorded domain is *not* the physical domain, and the $O(h^2)$ bias of (35) contaminates every control volume w_p and hence the weighted mean (15). Method 1, by integrating the smooth mapping (3), produces weights that converge to the true physical control volumes; this is why the default is `CELL_VOLUME_METHOD=1`.

This also explains the design of `VerifyPhysicalDomainVolume3D`. The check compares the summed weights against $V_{\text{discrete mesh}}$, the exact discrete-mesh volume formed from unique shoelace cells. For Method 0 the relative difference is required to pass the $< 10^{-12}$ tolerance — it *is* the discrete-mesh volume (telescoping, §5.1). For Method 1 the routine is expected to report a *nonzero* difference of the order of the boundary-curvature term (35): Method 1 deliberately departs from the chorded volume to recover the curved physical volume, and the routine additionally prints the analytic $L_X(L_Y L_Z - \int_0^{L_Y} h(y) dy)$ reference (Simpson, $N=10^4$) so that the discrete-vs-analytic gap can be read off directly. A “WARNING: differs from discrete mesh volume” for Method 1 is therefore the intended, physically correct behaviour, not a failure.

6.3 Error budget

Table 1 collects the leading error contributions. The geometric bias of Method 0 dominates and does not shrink under per-cell quadrature refinement (it converges only under mesh refinement, at the $O(h^2)$ rate); Method 1 removes it down to the degree-5 Lagrange reconstruction error $O(h^6)$ of (36) (the binding term—GL-3 integrates that reconstruction exactly, so the Gauss remainder (38) is non-binding here), which in turn falls to the floating-point floor of the parallel reduction by the finest grids. The corrector itself is algebraically exact (41), so the overall mass-closure error is set by the conditioning analysis of §6.1.

In summary, Method 1 replaces a fixed $O(h^2)$ geometric modelling error by an interpolation-limited $O(h^6)$ error (the degree-5 Lagrange reconstruction, which GL-3 integrates exactly) several orders of magnitude smaller and convergent under mesh refinement, while the median-dual partition of unity and the momentum-preserving rest-population shift make the resulting mass corrector algebraically exact, with a residual governed only by the well-conditioned, tree-summed parallel reduction.

Table 1: Error budget for the curvilinear mass-conservation corrector. h is the cell size, $n \sim 10^6$ the number of interior reduction nodes, $\varepsilon = 2.2 \times 10^{-16}$ the double-precision unit roundoff. “Per cell” errors refer to the area integral $A(j, k)$ of (34); “Global” aggregates over the $O(h^{-2})$ cross-section cells or the n nodes.

Source	Mechanism	Per-cell order	Global magnitude
Method 0 chord vs. curve	geometric, $O(h^2)$ boundary chord	$O(h^2)\kappa$	$O(h^2)$ chord bias (mesh-converging)
Lagrange-6 reconstruction (binding)	degree-5 interp. (39)	$O(h^6) J_{2D}^{(6)}$	$O(h^6)$ — dominant , sets the global rate
Method 1 GL-3 quadrature	Gauss remainder (38)	exact on the deg-5 reconstr.	non-binding (null on the reconstruction)
Partition-of-unity closure	$\sum_p w_p = V$ (40)	exact (telescoping)	$O(\varepsilon n/B)$
Correction algebra	one-shot shift (41)	exact	exact ($\rightarrow \langle p \rangle_V = 1$)
Reduction (naive sequential)	accumulation roundoff	—	$O(n\varepsilon) \approx 2.2 \times 10^{-10}$
Reduction (two-level: block tree + host seq.)	intra-block tree, then sequential host sum	—	$O(\varepsilon n/B) \approx 9 \times 10^{-13}$ ($B=256$)
Cross-rank MPI_Reduce	non-associative FP, fixed order	—	reproducible; $O(\varepsilon n/B)$ jitter on rank-count change

Table 2: Polynomial exactness of the tensor 3-point Gauss–Legendre rule ($\int_0^1 \xi^m d\xi$) used in the cell-volume scheme. Machine-zero error for $m \leq 5$ confirms the $2n - 1 = 5$ degree of exactness; the $m = 6$ error matches the analytic Gauss remainder $C f^{(6)}$ with $C = (3!)^4/[7(6!)^3] = 4.96 \times 10^{-7}$.

m	Exact $1/(m+1)$	GL-3 quadrature	Abs. error
0	1.0000000000000000	1.0000000000000000	0.000e+00
1	0.5000000000000000	0.5000000000000000	0.000e+00
2	0.3333333333333333	0.3333333333333334	5.551e-17
3	0.2500000000000000	0.2500000000000000	0.000e+00
4	0.2000000000000000	0.2000000000000000	0.000e+00
5	0.1666666666666667	0.1666666666666667	2.776e-17
6	0.1428571428571428	0.1425000000000000	3.571e-04
7	0.1250000000000000	0.1237500000000000	1.250e-03

7 Numerical experiments

The theory of §3–§6 is validated by an *independent, host-only verification harness* (a standalone C++ program, compiled with g++ 11.5.0 and `-O2 -std=c++17`, that re-implements the solver-identical quadrature, interpolation, and corrector arithmetic). The harness exercises three checks: (E1) polynomial exactness of the GL-3 rule, (E2) h -refinement convergence of the two area estimators on a curved, genuinely two-dimensional (non-separable) curvilinear map, and (E3) one-shot mass restoration with momentum preservation. It is deliberately decoupled from the GPU solver so that the mathematical properties are tested in isolation from the production code path.

7.1 E1: GL-3 polynomial exactness

Table 2 confirms the tensor GL-3 rule is exact to machine precision for polynomials of degree ≤ 5 and first errs at degree 6 by 3.571×10^{-4} , which reproduces the analytic Gauss remainder $C f^{(6)}$ (with $f = \xi^6$, $f^{(6)} = 720$, $C \cdot 720 = 3.571 \times 10^{-4}$) to all printed digits. The quadrature therefore has exactly the claimed $2n - 1 = 5$ degree of exactness.

7.2 E2: h -refinement convergence

Table 3 cleanly separates the two schemes. The test map is *non-separable*—its in-plane Jacobian J_{2D} carries a genuine $\xi\zeta$ cross term and varies in both the streamwise (ξ) and wall-normal (ζ) directions—so the tensor 6-point Lagrange reconstruction is genuinely exercised in the wall-normal

Table 3: Grid-convergence of the cross-section area on a genuinely two-dimensional, *non-separable* curvilinear y - z map: $y = \xi$, $z = \zeta(1 - a \sin \pi \xi) + (C - D \sin \pi \xi)(1 - \cos \pi \zeta)/\pi$, giving the in-plane Jacobian $J_{2D} = 1 - a \sin(\pi \xi) + C \sin(\pi \zeta) - D \sin(\pi \xi) \sin(\pi \zeta)$ ($a = 0.30$, $C = 0.20$, $D = 0.10$; $J_{2D} \in [0.7, 1.2] > 0$ everywhere; exact area $1 - 0.2/\pi - 0.4/\pi^2 = 0.8958095493$). The $\xi\zeta$ cross term makes J_{2D} vary in *both* directions, so the wall-normal Lagrange interpolation is genuinely exercised. Method 0 is the straight-chord Shoelace area; Method 1 is the Jacobian 3×3 Gauss–Legendre quadrature of the 6-point Lagrange interpolation of *sampled* nodal J_{2D} (solver-identical). Observed order $p = \log_2(e_h/e_{h/2})$.

N	h	Method 0 error	order	Method 1 error	order
8	0.125000	2.983e-03	–	2.636e-06	–
16	0.062500	7.443e-04	2.00	3.053e-08	6.43
32	0.031250	1.860e-04	2.00	4.268e-10	6.16
64	0.015625	4.649e-05	2.00	6.464e-12	6.04
128	0.007813	1.162e-05	2.00	1.040e-13	5.96
256	0.003906	2.905e-06	2.00	1.887e-15	†

† At $N=256$ Method 1 has reached the $\sim 2 \times 10^{-15}$ floating-point (reduction-rounding) floor; the error has dropped into the round-off band, so the apparent “order” there is no longer a genuine convergence rate. $N=128$ (1.040×10^{-13} , order 5.96) is still a genuine sixth-order step, so saturation first appears at $N=256$.

direction as well, not merely streamwise (a separable $J_{2D}(\xi)$ would make the ζ -interpolation trivially exact and leave that path untested). The straight-chord Shoelace (Method 0) converges at a textbook $O(h^2)$ (observed order 2.00 at every level) because its piecewise-linear facets chord the curved (one-signed) top wall (an Euler–Maclaurin trapezoid error), whereas the Jacobian GL-3 scheme (Method 1) converges at the predicted sixth order (observed orders 6.43, 6.16, 6.04, 5.96 from $N=8$ to $N=128$). The binding error is the degree-5 *Lagrange reconstruction* of the sampled metric, which GL-3 integrates exactly, so the reconstruction—not the quadrature—sets the $O(h^6)$ rate. At $N=256$ the Method 1 error has fallen to 1.887×10^{-15} , i.e. into the $\sim 2 \times 10^{-15}$ reduction-rounding floor (marked †); $N=128$ (1.040×10^{-13} , order 5.96) is still a genuine sixth-order step, so the saturation first appears at $N=256$, one level beyond the last clean rate. Because the analytic map keeps $J_{2D} \in [0.7, 1.2]$ strictly positive everywhere, the solver’s `isfinite/≤0` Shoelace fallback is never triggered in this test (fallback count = 0): it is a never-exercised safety path here, which we report honestly as untested rather than claim as validated. This confirms the error-budget claim that Method 1 replaces a fixed $O(h^2)$ geometric bias with a convergent, interpolation-limited $O(h^6)$ error several orders of magnitude smaller.

7.3 E3: one-shot mass restoration and momentum preservation

On a 16×16 curvilinear patch, median-dual weights were built with each volume method, the nodal density was perturbed, and the correction $\delta\rho = 1 - \langle\rho\rangle_V$ was injected into the rest population f_0 . For both methods the perturbed mean $\langle\rho\rangle_V = 0.99700000$ ($\delta\rho = +3.0 \times 10^{-3}$) returned to unity after a single application to within the reduction-rounding floor: Method 0 (Shoelace, $V = 0.8965538$) gave $|1 - \langle\rho\rangle_V^{\text{new}}| = 1.110 \times 10^{-15}$ and Method 1 (Jacobian GL-3, $V = 0.8958096$, agreeing with the analytic area 0.8958095 to the $\sim 3 \times 10^{-8}$ Method-1 area error at this $N=16$ resolution) gave 2.220×10^{-16} . The Method-1 path again recorded zero Shoelace fallbacks (the smooth $J_{2D} > 0$ never trips the `isfinite/≤0` guard). A separate momentum test injected $\delta\rho = 0.0123456789$ into f_0 of a D3Q19 stencil ($\mathbf{e}_0 = \mathbf{0}$): the density rose by exactly $\delta\rho$ while the momentum $\sum_q \mathbf{e}_q f_q$

stayed at $(0,0,0)$ *exactly*. This validates the one-shot algebraic exactness (41) and confirms that the rest population is the correct, momentum-silent mass carrier. The harness artefacts reside under `docs/numexp/` (`verify_gl3.cpp`, `gl3_results.txt`).

7.4 E4: partition-of-unity on the production grid — a bit-exactness audit

Experiments E1–E3 exercise the quadrature on analytic patches. E4 closes the loop on the *actual* $\text{Re}_h=2800$ production grid (`J_Frohlich/adaptive_3.finegrid_I513_J257_s0.950000.dat`; $N_x \times N_y \times N_z = 257 \times 513 \times 257$, hence $256 \times 512 \times 256 = 3.36 \times 10^7$ cells) and answers one question with hard data: is the start-up control volume $\Sigma_w = \sum_p w_p$ (the $\frac{1}{8}$ -split dual weights of `InitializeMassCorrectionWeights`) *bit-exactly* equal to the discrete main-field volume $V_{\text{disc}} = \sum_c V_c$ (`ComputeGlobalDiscreteShoelaceVolume3D`)? A self-contained host harness (`docs/numexp/test_volume_partition.cpp`, single process, $jp=1$ whole-domain so the per-rank slice equals the global array) reads the real grid through a byte-faithful copy of the solver’s `GenerateMesh_X / ReadExternalGrid_YZ` and evaluates both sums in the solver’s exact loop order and with its exact constants (0.125, the Shoelace formula). The full transcript is in `docs/numexp/partition_results.txt`.

The four bit-exactness metrics (Method 0). On the production grid,

$$\begin{aligned} V_{\text{disc}} &= 1.14358939042152798 \times 10^2, & \Sigma_w &= 1.14358953365302284 \times 10^2, \\ \text{(i) exact (==) : } &\mathbf{FALSE}, & \text{(ii) ULP gap} &= 1,007,902,042 \approx 1.0 \times 10^9, \\ \text{(iii) } |\Sigma_w - V_{\text{disc}}| &= 1.432 \times 10^{-5}, & \text{(iv) rel} &= 1.252 \times 10^{-7}. \end{aligned} \tag{43}$$

The two doubles are **not** bit-identical (BIT-EXACT: NO), and, decisively, the gap ($\sim 10^{-7}$ relative, $\sim 10^9$ ULP) sits five to eight orders of magnitude *above* the IEEE rounding floor. It is therefore *not* mere summation non-associativity — there is a genuine, geometric cause, isolated next.

Cause: the streamwise periodic seam is not bit-closed. The $\frac{1}{8}$ dual telescopes *exactly* in two of three directions: the spanwise x wrap is closed by the explicit remap $i=3 \rightarrow N_x-5$ onto a *real* interior cell (and the in-plane area is x -independent, so all 256 spanwise copies are bit-identical), and the wall-normal k stencil is one-sided at the walls with no periodic image required. The streamwise y direction, however, is closed only *implicitly*, through the periodic ghost: the seam node ($j=3$, Fröhlich $I=0$) draws its left dual cell from the wrapped ghost cell $j_{\text{cell}}=2$, whereas V_{disc} counts the real last cell $j_{\text{cell}}=514$ (between nodes $I=512$ and the seam). These two cells are translates by L_y *only if* the grid is bit-periodic. The production Fröhlich grid is not: nodes $I=0$ and $I=512$ differ from exact periodic images by

$$\max_k |y_{I=0} + L_y - y_{I=512}| = 3.285 \times 10^{-6}, \quad \max_k |z_{I=0} - z_{I=512}| = 0,$$

so the seam ghost area and the real last-cell area differ by up to 2.62×10^{-4} per wall-normal layer. Carrying that mismatch through the $\frac{1}{2}$ seam coefficient $\times (N_x-1) \Delta x$ predicts a closure defect of 1.432306×10^{-5} , which reproduces the *measured* $\Sigma_w - V_{\text{disc}} = 1.432315 \times 10^{-5}$ to a relative 6.5×10^{-6} . The seam thus accounts for essentially the entire gap.

Summation order (MPI non-associativity) is a sub-effect. Re-summing the same per-cell values in the real run’s $jp=32$ grouping (32 contiguous streamwise rank-blocks of 16, reduced in rank order to mimic `MPI.Reduce`) shifts each total only in its last bits: Σ_w moves by 28,313 ULP

contribution to $\Sigma_w - V_{\text{disc}}$	abs	rel	ULP
streamwise grid seam non-closure (geometry)	1.43×10^{-5}	1.25×10^{-7}	1.0×10^9
$jp=1$ vs $jp=32$ summation order (MPI)	$\sim 4 \times 10^{-10}$	$\sim 3 \times 10^{-12}$	$\sim 2\text{--}3 \times 10^4$
IEEE rounding floor (closed-seam control)	9.3×10^{-11}	8.1×10^{-13}	6.5×10^3

Table 4: Three-tier decomposition of the Method-0 partition-of-unity gap on the production grid. The measured 1.25×10^{-7} closure defect is *geometry*, dominated by the grid’s streamwise seam ($\sim 3 \times 10^{-6}$ non-closure); floating-point non-associativity (summation/MPI order) is the $\sim 10^{-12}$ sub-effect, and the bit-closed mesh recovers the $\sim 10^{-13}$ rounding floor.

(4.02×10^{-10} abs, 3.52×10^{-12} rel) and V_{disc} by 21,543 ULP (3.06×10^{-10} abs, 2.68×10^{-12} rel) — the genuine floating-point non-associativity caveat, of the order $n\varepsilon$ anticipated in (42), and ~ 5 orders smaller than the seam.

Closed-seam control: the Proposition holds to rounding. Forcing perfect periodic closure (node $I=0 := \text{node } I=512 - L_y$) and recomputing *both* sums on the closed grid collapses the discrepancy to the rounding floor:

$$|\Sigma_w - V_{\text{disc}}|_{\text{closed}} = 9.27 \times 10^{-11}, \quad \text{rel} = 8.11 \times 10^{-13} (< 10^{-12}), \quad 6,522 \text{ ULP},$$

i.e. the real-grid 1.25×10^{-7} falls to 8.1×10^{-13} once the seam is bit-closed. This is the clean experimental statement of Proposition 1: $\sum_p w_p = \sum_c V_c$ is an *exact-arithmetic* identity that holds to IEEE-double rounding ($\sim 10^{-13}$) — *not* bitwise — on a bit-closed periodic mesh, and is degraded only by the grid’s own seam-closure tolerance, not by the algorithm.

Comparison to the analytic hill volume. The continuum reference $V_{\text{phys}} = L_x(L_y L_z - \int_0^{L_y} h \, dy) = 1.143590335916 \times 10^2$ (Simpson, $N=10^4$, mirroring `VerifyPhysicalDomainVolume3D`) is approached by both discrete volumes at the $O(h^2)$ chord level: V_{disc} differs from V_{phys} by 8.27×10^{-7} (rel) and Σ_w by 7.02×10^{-7} — the expected geometric chord error of the curved hill, an order of magnitude larger than the seam defect and three orders larger than the FP floor.

Method 1 (the default) departs by design. The shipped configuration is `CELL_VOLUME_METHOD=1`. Reproducing the real metric-term FD stencil (`ComputeMetricTerms_Full`; six-order Fornberg in k , six-order central in j , periodic- j ghost) to obtain J_{2D} on the production grid and applying the GL-3 / Lagrange-6 weight sum gives $\Sigma_w^{\text{M1}} = 1.14359016044 \times 10^2$, which departs from V_{disc} by 6.73×10^{-7} (rel) — a *deliberate* $O(h^2)$ difference, not a rounding effect, because Method 1 integrates the curved J_{2D} rather than its straight chords (per-cell $|J - \text{Shoe}|/\text{Shoe}$ up to 1.78×10^{-2} , mean 9.21×10^{-5} ; zero Shoelace fallbacks, the smooth $J_{2D} > 0$ never tripping the guard). Consistently with E2 and §5.1, Method 1 lies *closer* to the continuum volume: $|\Sigma_w^{\text{M1}} - V_{\text{phys}}|/V_{\text{phys}} = 1.53 \times 10^{-7}$ versus Method 0’s 8.27×10^{-7} .

Reconciliation and verdict. The audit confirms the precise status of the partition-of-unity identity on real hardware. *It is not bitwise* (`==` is FALSE, $\sim 10^9$ ULP): Σ_w equals the discrete main-field volume to a relative 1.25×10^{-7} , dominated by the production grid’s streamwise seam non-closure ($\sim 3 \times 10^{-6}$ in y) and *not* by the $\sim 10^{-12}$ floating-point non-associativity of the $\frac{1}{8}$ -split re-summation (the latter amplified, as a caveat, by `MPI_Reduce` rank order). Because 1.25×10^{-7} (Method 0) — and 6.73×10^{-7} for the default Method 1 — both exceed the solver’s documented

10^{-12} gate, `VerifyPhysicalDomainVolume3D` emits its `WARNING`, not `PASS`, on this grid; the warning is faithful, reporting a real ($O(h^2)$ /seam-level) geometric reality, not a defect of the weight algebra. The closed-seam control ($\text{rel } 8.1 \times 10^{-13} < 10^{-12}$, Table 4) shows Proposition 1 holds to rounding the instant the mesh is bit-closed, exactly as the exact-arithmetic proof predicts.

8 Implementation: mapping the mathematics to source code

This section maps every symbol of the volume-weighted mass-correction formulation to the concrete C++/CUDA that realizes it. All references are to the Periodic-Hill D3Q19 GILBM solver (`Edit13_2800ITBLBM`); line numbers are 1-indexed into the cited files.

8.1 Theory $\leftrightarrow$ code correspondence

Symbol / step	Realizing function	Location
$V_{\text{cell}} = \Delta x_i A_{\text{quad}}$ (Shoelace)	<code>MassCorrectionCellVolume</code>	<code>evolution.h:82–103</code>
$w_p = \frac{1}{8} \sum_{c \in p} V_{\text{cell}}$ (median-dual)	<code>InitializeMassCorrectionWeights</code>	<code>evolution.h:105–172</code>
GL-3 nodes ξ_a , weights W_a on $[0, 1]$	<code>GL3_nodes</code> , <code>GL3_weights</code>	<code>evolution.h:180–189</code>
Degree-5 Lagrange basis $\ell_m(x)$	<code>Lagrange6Weights</code>	<code>evolution.h:191–204</code>
Centered/clamped 6-wide stencil start	<code>SelectStencilStart</code>	<code>evolution.h:206–213</code>
$\tilde{J}_{2D} = \sum_{m,n} \ell_m \ell_n J_{2D}$	<code>InterpolateJ2D_Lagrange6</code>	<code>evolution.h:215–231</code>
$A_{\text{jac}} \approx \sum_{a,b} W_a W_b \tilde{J}_{2D}$	<code>ComputeJacobianMassCorrectionWeights</code>	<code>evolution.h:233–362</code>
$V_{\text{mesh}} = \sum_c V_{\text{cell}}$ (discrete check)	<code>ComputeGlobalDiscreteShoelaceVolume3D</code>	<code>evolution.h:364–383</code>
$\text{RelErr} < 10^{-12}$ vs. discrete mesh	<code>VerifyPhysicalDomainVolume3D</code>	<code>evolution.h:390–422</code>
$\sum_p w_p \rho_p$ (interior reduction)	<code>ReduceRhoWeightedSum_Kernel</code>	<code>evolution.h:47–80</code>
$\langle \rho \rangle_V = (\sum_p w_p \rho_p) / V$	<code>ComputeVolumeWeightedRhoAverageRoot</code>	<code>evolution.h:424–443</code>
$\delta \rho = 1 - \langle \rho \rangle_V + \text{MPI.Bcast}$	<code>UpdateVolumeWeightedMassCorrection</code>	<code>evolution.h:445–453</code>
$\rho \leftarrow \rho + \delta \rho$, $f_0 \leftarrow f_0 + \delta \rho$	fused collision kernel (apply site)	<code>gilbm/evolution_gilbm/1.algorithm1.h:407–411</code>
$V = \sum_p w_p$ via <code>MPI.Allreduce</code>	<code>rho_cv_global_volume</code>	<code>evolution.h:149,333</code>
$J_{2D} = \det \partial(y, z) / \partial(\xi, \zeta)$	<code>ComputeMetricTerms_Full</code>	<code>gilbm/metric_terms.h:160–161</code> (#incl. at <code>main.cu:233</code>)
ghost-row exchange of J_{2D} (6-pt stencil)	<code>MPI.Sendrecv</code> loop	<code>main.cu:743–749</code> (list :741)
method selection + verification wiring	init block	<code>main.cu:763–769</code>
compile-time method switch (default 1)	<code>CELL_VOLUME_METHOD</code>	<code>variables.h:342–347</code>

Table 5: Each mathematical object maps to exactly one function/line range in the source.

8.2 Key kernels (verbatim)

Note. In the listings below the executable lines are reproduced verbatim from the solver source; only the inline comments have been translated from the original Traditional-Chinese for readability (e.g. the apply-site comment in `gilbm/evolution\_gilbm/1.algorithm1.h:407–411`).

8.2.1 Shoelace cell volume

The planar quadrilateral area uses the surveyor (Gauss) formula on the four corners (y, z) at (j, k) , $(j+1, k)$, $(j+1, k+1)$, $(j, k+1)$, multiplied by the spanwise width.

```

1 static inline double MassCorrectionCellVolume(const int i_cell, const int j_cell, const int k_cell)
2 {
3     const double dx = x_h[i_cell + 1] - x_h[i_cell];
4
5     const int jk00 = j_cell * NZ6 + k_cell;
6     const int jk10 = (j_cell + 1) * NZ6 + k_cell;
7     const int jk11 = (j_cell + 1) * NZ6 + (k_cell + 1);
8     const int jk01 = j_cell * NZ6 + (k_cell + 1);
9

```

```

10  const double y0 = y_2d_h[jk00], z0 = z_h[jk00];
11  const double y1 = y_2d_h[jk10], z1 = z_h[jk10];
12  const double y2 = y_2d_h[jk11], z2 = z_h[jk11];
13  const double y3 = y_2d_h[jk01], z3 = z_h[jk01];
14
15  const double area_yz = 0.5 * fabs(
16      y0 * z1 - z0 * y1
17      + y1 * z2 - z1 * y2
18      + y2 * z3 - z2 * y3
19      + y3 * z0 - z3 * y0);
20
21  return fabs(dx) * area_yz;
22 }

```

Listing 1: Shoelace cell volume (evolution.h:82–103).

Annotation. The four jk^* indices fetch the corner coordinates from the y - z section arrays y_2d_h/z_h . area_yz is the cyclic Shoelace sum $\frac{1}{2} \left| \sum y_a z_{a+1} - z_a y_{a+1} \right|$, the *exact* area of the straight-chord quadrilateral. The return value $|\Delta x_i| \cdot A_{\text{quad}}$ realizes $V_{\text{cell}} = |\Delta x_i| A_{\text{quad}}$. Because adjacent quads share edges that cancel, $\sum_c V_{\text{cell}}$ telescopes to the exact discrete-mesh polygon volume; this is precisely the quantity `VerifyPhysicalDomainVolume3D` checks against (printing PASS) to $< 10^{-12}$.

8.2.2 GL-3 data, Lagrange basis, and J_{2D} reconstruction

```

1  static const double GL3_nodes[3] = {
2      0.5 * (1.0 - 0.7745966692414834), // (1 - sqrt(3/5))/2 ~ 0.1127
3      0.5,
4      0.5 * (1.0 + 0.7745966692414834) // (1 + sqrt(3/5))/2 ~ 0.8873
5  };
6  static const double GL3_weights[3] = {
7      5.0 / 18.0, // ~ 0.27778
8      8.0 / 18.0, // ~ 0.44444
9      5.0 / 18.0
10 };
11
12 static inline void Lagrange6Weights(double x, int start, double w[6])
13 {
14     for (int m = 0; m < 6; m++) {
15         double L = 1.0;
16         double xm = (double)(start + m);
17         for (int r = 0; r < 6; r++) {
18             if (r != m) {
19                 double xr = (double)(start + r);
20                 L *= (x - xr) / (xm - xr);
21             }
22         }
23         w[m] = L;
24     }
25 }
26
27 static inline double InterpolateJ2D_Lagrange6(
28     double xi_pos, double zeta_pos, int sj, int sk,
29     const double *J_2D, int NZ6_local)
30 {
31     double wj[6], wk[6];
32     Lagrange6Weights(xi_pos, sj, wj);

```

```

33   Lagrange6Weights(zeta_pos, sk, wk);
34
35   double result = 0.0;
36   for (int m = 0; m < 6; m++) {
37       double row_sum = 0.0;
38       for (int n = 0; n < 6; n++)
39           row_sum += wk[n] * J_2D[(sj + m) * NZ6_local + (sk + n)];
40       result += wj[m] * row_sum;
41   }
42   return result;
43 }

```

Listing 2: GL-3 nodes/weights, degree-5 Lagrange basis, and tensor reconstruction (evolution.h:180–231).

Annotation. The constant 0.7745966692414834 is $\sqrt{3/5}$, so `GL3_nodes` are the three Gauss–Legendre abscissae mapped onto $[0, 1]$, $\{(1 - \sqrt{3/5})/2, \frac{1}{2}, (1 + \sqrt{3/5})/2\}$, with weights $\{\frac{5}{18}, \frac{8}{18}, \frac{5}{18}\}$ written exactly as 5.0/18.0 etc. GL-3 is exact for polynomials of degree ≤ 5 per direction. `Lagrange6Weights` builds the six cardinal degree-5 basis functions $\ell_m(x) = \prod_{r \neq m} (x - x_r)/(x_m - x_r)$ on integer nodes $\{start, \dots, start+5\}$. `InterpolateJ2D_Lagrange6` forms the tensor product $\tilde{J}_{2D} = \sum_m w_j[m] \sum_n w_k[n] J_{2D}[(s_j+m, s_k+n)]$. Since the reconstruction is degree-5 and GL-3 integrates degree-5 exactly, the interpolation does *not* reduce the quadrature order.

8.2.3 Jacobian-GL nodal weights (core triple loop)

```

1  for (int j = 3; j < NYD6 - 4; j++) {
2  for (int k = 3; k < NZ6 - 3; k++) {
3  for (int i = 3; i < NX6 - 4; i++) {
4      const int idx = j * NX6 * NZ6 + k * NX6 + i;
5
6      const int i_cells[2] = { (i == 3) ? (NX6 - 5) : (i - 1), i }; // periodic x wrap
7      const int j_cells[2] = { j - 1, j };
8      int k_cells[2]; int nk_cells = 0;
9      if (k > 3) k_cells[nk_cells++] = k - 1; // one-sided near wall
10     if (k < NZ6 - 4) k_cells[nk_cells++] = k;
11
12     double weight = 0.0;
13     for (int jj = 0; jj < 2; jj++) {
14     for (int ii = 0; ii < 2; ii++) {
15     for (int kk = 0; kk < nk_cells; kk++) {
16         const int jc = j_cells[jj];
17         const int kc = k_cells[kk];
18         const double dx = x_h[i_cells[ii] + 1] - x_h[i_cells[ii]];
19
20         int sj, sk;
21         bool sj_ok = SelectStencilStart(jc, j_lo_J, j_hi_J, &sj);
22         bool sk_ok = SelectStencilStart(kc, k_lo_J, k_hi_J, &sk);
23
24         double vol_jac; bool used_fallback = false;
25         if (sj_ok && sk_ok) {
26             double area_jac = 0.0;
27             for (int a = 0; a < 3; a++) {
28                 for (int b = 0; b < 3; b++) {
29                     double xi_pos = (double)jc + GL3_nodes[a];
30                     double zeta_pos = (double)kc + GL3_nodes[b];
31                     double J_val = InterpolateJ2D_Lagrange6(xi_pos, zeta_pos, sj, sk, J_2D, NZ6);
32                     if (!std::isfinite(J_val) || J_val <= 0.0) { used_fallback = true; break; }

```

```

33         area_jac += GL3_weights[a] * GL3_weights[b] * J_val;
34     }
35     if (used_fallback) break;
36 }
37 vol_jac = fabs(dx) * area_jac;
38 } else { used_fallback = true; }
39
40 if (used_fallback) {
41     vol_jac = MassCorrectionCellVolume(i_cells[ii], jc, kc); // Shoelace fallback
42     local_fallback_count++;
43 }
44
45 double vol_shoe = MassCorrectionCellVolume(i_cells[ii], jc, kc);
46 if (vol_shoe > 0.0) {
47     double rd = fabs(vol_jac - vol_shoe) / vol_shoe;
48     if (rd > local_max_rel_diff) local_max_rel_diff = rd;
49     local_sum_rel_diff += rd; local_cell_count++;
50 }
51 weight += 0.125 * vol_jac;
52 }}}
53 rho_cv_weight_h[idx] = weight;
54 local_weight_sum += weight;
55 }}}

```

Listing 3: Median-dual assembly with GL-3 cells and Shoelace fallback (evolution.h:233–362, excerpt).

Annotation. The loop visits each interior unique node idx and accumulates the median-dual weight $w_p = \frac{1}{8} \sum_{c \in p} V_{\text{cell}}$ via the $0.125 * \text{vol_jac}$ contribution of up to eight surrounding cells (jj, ii, kk). The periodic x wrap is the ternary $(i == 3) ? (NX6 - 5) : (i - 1)$; near-wall k degrades to a one-sided single cell (nk_cells can be 1). For each cell the inner 3×3 loop forms $A_{jac} = \sum_{a,b} W_a W_b \tilde{J}_{2D}(jc + \xi_a, kc + \zeta_b)$ with $\tilde{J}_{2D}$ from `InterpolateJ2DLagrange6`, and the cell volume is $|\Delta x| \cdot A_{jac}$. Any non-finite or non-positive reconstructed J trips `used_fallback`, which substitutes the exact Shoelace volume and increments `local_fallback_count`. Independently, every cell records the per-cell relative difference $|V_{jac} - V_{shoe}|/V_{shoe}$ feeding the [MASS-CORR] max/mean report.

8.2.4 Average, correction, and the rest-population apply site

```

1 // evolution.h:445-453 --- delta_rho = 1 - <rho>_V, broadcast to all ranks
2 static inline void UpdateVolumeWeightedMassCorrection()
3 {
4     const double rho_avg = ComputeVolumeWeightedRhoAverageRoot(); // (sum w*rho)/V on root
5     if (myid == 0) {
6         rho_modify_h[0] = 1.0 - rho_avg;
7     }
8     MPI_Bcast(rho_modify_h, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
9     CHECK_CUDA( cudaMemcpy(rho_modify_d, rho_modify_h, sizeof(double), cudaMemcpyHostToDevice) );
10 }
11
12 // gilbm/evolution_gilbm/1.algorithm1.h:407-411 --- apply at every interior node, in the fused
13 // kernel
14 // -- STEP 1.5: Macroscopic (mass correction) --
15 if (i < NX6 - 4 && j < NYD6 - 4) {
16     rho_stream += rho_modify[0];
17     f_arr[0] += rho_modify[0]; // correct f_arr[0] held in register

```

Listing 4: Correction scalar, broadcast, and the rest-population apply site (`evolution.h:445–453; gilbm/evolution_gilbm/1.algorithm1.h:407–411`).

Annotation. `rho_modify_h[0] = 1.0 - rho_avg` implements $\delta\rho = 1 - \langle\rho\rangle_V$ on the root rank; the `MPI_Bcast` distributes the single scalar so every rank applies an identical shift. At the apply site the density moment receives $\rho \leftarrow \rho + \delta\rho$ and the *rest* population receives $f_0 \leftarrow f_0 + \delta\rho$. Because $\mathbf{e}_0 = (0, 0, 0)$, the injected momentum is $\Delta(\rho u_\alpha) = \sum_q e_{q,\alpha} \Delta f_q = 0$: mass is injected, momentum is exactly preserved. The shift is spatially uniform, so the new average is $\langle\rho\rangle_V + \delta\rho = 1$ in a single application (the only residual is floating-point rounding of the parallel reduction)—a discrete “mass governor” with unit feedback gain.

8.3 Implementation notes

Compile-time method switch and #error guard. `variables.h:342–347` defines the selector with a hard guard:

```
1 #ifndef CELL_VOLUME_METHOD
2 #define CELL_VOLUME_METHOD 1
3 #endif
4 #if CELL_VOLUME_METHOD != 0 && CELL_VOLUME_METHOD != 1
5 #error "CELL_VOLUME_METHOD must be 0 (Shoelace) or 1 (Jacobian 3x3 GL)"
6 #endif
```

Listing 5: Method selector with compile-time guard (`variables.h:342–347`).

The default is 1 (Jacobian-GL). The entire `ComputeJacobianMassCorrectionWeights` body is wrapped in `#if CELL_VOLUME_METHOD == 1` (`evolution.h:233–362`), so Method 0 builds compile only `InitializeMassCorrectionWeights`. `main.cu:763–769` wires the choice: under Method 1 it calls `ComputeJacobianMassCorrectionWeights(J_2D_h, &shoelace_vol)` then verifies *both* the Shoelace and Jacobian-GL volumes; otherwise it verifies the single Shoelace volume.

Shoelace fallback path and diagnostics. Method 1 falls back to the exact Shoelace area whenever the 6-wide Lagrange stencil is unavailable (`SelectStencilStart` returns `false` near a partition edge: it needs `max_start = hi - 5` with `max_start - lo >= 0`) or whenever a reconstructed J_{2D} value is non-finite or ≤ 0 . The root rank prints the audit block (`evolution.h:348–358`): Shoelace global volume, Jacobian global volume, $\Delta(\text{Jac-Shoe})/\text{Shoe}$, Per-cell max rel diff, Per-cell mean rel diff, and Fallback to Shoelace = N cells. `VerifyPhysicalDomainVolume3D` additionally checks $\sum_p w_p$ against the telescoped discrete-mesh volume `ComputeGlobalDiscreteShoelaceVolume3D`. It prints PASS when `rel_err_discrete < 1e-12` else a WARNING, and reports the analytic $L_X(L_Y L_Z - \int_0^{L_Y} h \, dy)$ Hill integral (Simpson, $N = 10000$) as a reference only.

MPI reductions. Total control volume $V = \sum_p w_p$ is a collective `MPI_Allreduce` (`evolution.h:149` for Method 0, :333 for Method 1) into `rho_cv_global_volume` so every rank holds V . The weighted mass $\sum_p w_p \rho_p$ is gathered by an `MPI_Reduce` to root in `ComputeVolumeWeightedRhoAverageRoot` (`evolution.h:439`); root forms $\langle\rho\rangle_V$ and $\delta\rho$, then `MPI_Bcast` (`evolution.h:451`) distributes the scalar. The 6-point Lagrange stencil reaches into neighbouring streamwise partitions, so the seven-array list at `main.cu:741` includes `J_2D_h` and the ghost-row `MPI_Sendrecv` loop at `main.cu:743–749` exchanges `{xi_y_h, xi_z_h, zeta_y_h, zeta_z_h, y_2d_h, z_h, J_2D_h}`; the Jacobian-weight code then allows j -stencils into the ghost rows (`j_lo_J = 0`, `j_hi_J = NYD6-1`) while keeping k -stencils on physical wall-normal nodes (`k_lo_J = 3`, `k_hi_J = NZ6-4`).

The f_0 rest-population injection invariant. The reduction kernel `ReduceRhoWeightedSum_Kernel` (`evolution.h:47–80`) sums $\rho_p w_p$ over exactly the interior unique-node box that the corrector later modifies ($i = 3..NX6-5$, $j = 3..NYD6-5$, $k = 3..NZ6-4$), guaranteeing the averaged and corrected sets coincide. With D3Q19 ($N_Q = 19$, $W_0 = 1/3$, $\mathbf{e}_0 = \mathbf{0}$) and the current `FORCE_HERMITE_ORDER = 2` (full Guo, whose first-order density moment vanishes), the body force injects no net mass; the corrector’s $f_0 \ += \delta\rho$ is the sole, momentum-neutral channel that drains the accumulated compressibility drift back to $\langle\rho\rangle_V = 1$.

9 Verification of the collision operator

For the volumetric corrector to be the *sole* controlled mass source, the collision operator and the body force must inject no spurious mass. We verified this for the MRT collision on the conserved-moment basis of the D3Q19 transform M , whose rows were checked numerically (not by annotation): row $k=0$ is the all-ones density vector and rows $k=3, 5, 7$ equal the lattice-velocity components e_x, e_y, e_z . The corresponding relaxation rates are pinned to zero, $s_0=s_3=s_5=s_7=0$, so the precomputed projection $K = M^{-1}SM$ leaves these moments untouched; we confirmed $\max_{k \in \{0,3,5,7\}} \|M_k K\|_\infty = 1.66 \times 10^{-15}$, i.e. the collision exactly conserves mass and momentum to machine precision. The Guo body force, injected at second Hermite order $F_i = W_i \mathcal{F}[3(c_y - v) + 9(\mathbf{c} \cdot \mathbf{u})c_y]$ via the split tables $(F_{\text{proj}}, F_{\text{proj}}^{u,v,w})$ with the half-force factor $(1 - s_k/2)=1$, satisfies $\sum_i F_i = \mathcal{O}(10^{-17})$ (zero net mass source) while delivering exactly the prescribed streamwise momentum $\sum_i F_i c_y = 1$. Consequently the lattice collision contributes no spurious mass, and the volumetric density corrector of §3 remains the sole controlled source of mass in the scheme. The full machine-checked record is reproduced in Appendix A.

10 Discussion and limitations

10.1 Discrete self-consistency, not field fidelity, is the correctness criterion

The seam audit of §7.4 invites a question of principle that we now settle: against *what* reference is the volume-weighted corrector to be judged correct? The operative criterion is *internal self-consistency on the solver’s own discrete grid*—not agreement with the true straight-boundary physical domain, and not even exact partition-of-unity closure. We make this precise.

The corrector is unconditionally self-consistent. The governor of §3.3 forms $\langle\rho\rangle_V$ from (15) using the *same* nodal weights w_p in numerator and denominator. Consequently, for any spatially uniform field $\rho_p \equiv c$,

$$\rho_p \equiv c \implies \langle\rho\rangle_V = \frac{\sum_p \rho_p w_p}{\sum_p w_p} = \frac{c \sum_p w_p}{\sum_p w_p} = c, \quad (44)$$

to floating-point rounding and *independently of the actual values of the weights w_p* . In particular $\rho_p \equiv 1$ yields $\langle\rho\rangle_V = 1$ and hence $\delta\rho = 1 - \langle\rho\rangle_V = 0$ by (16): the corrector injects nothing into a uniform field, and the one-shot map (21) has already reached its fixed point (22). Crucially, this identity holds *unconditionally*—regardless of whether $\Sigma_w = \sum_p w_p$ equals the discrete cell sum $\sum_c V_c$, and regardless of whether either equals the true physical volume V_{phys} . It is a property of the *algebraic form* of (15), not of any geometric accuracy of w_p .

Three distinct notions of “volume agreement” must not be conflated. The discussion of §4.3 and §7.4 touches three logically independent statements, only the first of which bears on correctness:

- (i) *Internal self-consistency*: numerator and denominator of (15) share the weights w_p , so a uniform field is reproduced exactly and no spurious correction is injected. This is (44); it holds *unconditionally*, for *any* positive weights.
- (ii) *Partition-of-unity closure*: $\Sigma_w = \sum_p w_p$ equals $\sum_c V_c = V_{\text{mesh}}$ on the solver’s own grid—Proposition 1, equation (32). This is an exact-arithmetic identity that holds to IEEE-double rounding *if and only if* the grid’s periodic seam is bit-closed; it is exactly the quantity `VerifyPhysicalDomainVolume3D` measures, and exactly what the seam audit (43) probes.
- (iii) *Field fidelity*: Σ_w equals V_{phys} , the volume of the continuum straight-boundary periodic-hill domain. This is *never* exact—the discrete boundary is a curve, and the interior outlet rows land at, e.g., $y \approx 8.9999967$ rather than $y = L_y$ (the read-time normalization pins the anchor row exactly). The discrepancy is the $O(h^2)$ chord error of the curved hill plus the single-precision streamwise seam non-closure.

Only (i) is required for correctness. Property (i) is *necessary and sufficient* for the corrector to do its job: hold the volume-weighted mean density at unity without injecting a spurious source. Property (ii) is a desirable consistency and diagnostic property—it certifies that Σ_w is a faithful discrete mesh volume—but the corrector is self-consistent and mass-conserving whether or not (ii) holds exactly, because (44) never invokes $\sum_c V_c$. Property (iii) is a statement of *geometric accuracy*—how faithfully the discrete domain approximates the continuum—and therefore bears on *solution* accuracy, not on mass-conservation *correctness*. Mass is conserved on the grid the solver is actually given (curved boundary, single-precision provenance and all): the corrector holds $\langle \rho \rangle_V$ at unity with respect to its *own* control volumes w_p , which is the only mass that the discrete dynamics either sees or can conserve.

The seam discrepancy is a geometric diagnostic, not a defect. It follows that the VOL-CHECK discrepancy reported in the E4 audit (Σ_w versus $\sum_c V_c$, relative $\sim 10^{-7}$, (43)) is a *geometric diagnostic of streamwise periodic closure*—it flags that the production grid’s outlet is not bit-closed onto its inlet—and is *not* a mass-conservation defect. The corrector never reads $\sum_c V_c$; it reads Σ_w in both numerator and denominator, so the seam gap cancels identically in (15) and cannot bias $\delta\rho$. Closing the seam—E4’s control, node $I=0 := \text{node } I=512 - L_y$, i.e. making the two boundary edges exact periodic images using the grid’s own coordinates—makes (ii) exact: it collapses the discrepancy to the rounding floor (PASS, §7.4, Table 4) and additionally renders the grid bit-periodic. But this is a cleanliness and diagnostic refinement, *not* a correctness fix: the corrector was already self-consistent and mass-conserving before closure. The grid is not “wrong”—it is a faithful $O(h^2)$ /seam-level discrete approximation of the continuum hill—and the corrector’s correctness is established by (44) alone, leaving the seam a question of grid closure and geometry only.

Why volume weighting is not optional. The central message is structural: on a stretched body-fitted mesh, the only mesh-invariant target for a density governor is the volume-weighted mean $\langle \rho \rangle_V$. Regulating the unit-weight sum $\widetilde{M}$ of (12) is not a milder approximation but a qualitatively wrong control objective—it pins a metric-biased functional and feeds a spurious, mesh-dependent momentum source. The median-dual weights make the correct objective computable at the cost of one precomputed weight array and one extra reduction per step.

Cost. The corrector adds a single global reduction ($\sum_p w_p \rho_p$), a scalar broadcast, and a per-node add fused into the existing collision kernel. The weight array w_p and the volume V are built once at start-up; the GL-3/Lagrange machinery is therefore amortised entirely into initialisation and does not enter the time-stepping cost.

Limitations. (i) The correction is *uniform*: it removes the global mean drift but does not redistribute mass spatially, so a genuinely spatially-structured mass error (e.g. from a defective boundary condition) would not be cured—only its global mean. (ii) The one-shot exactness (41) holds for the volume-weighted mean; the residual is the non-associative parallel-reduction rounding of (42), which makes $\delta\rho$ reproducible only at fixed rank count and fixed reduction tree. (iii) Method 1’s sixth-order accuracy presumes a *smooth* metric J_{2D} ; near a geometric singularity the degree-5 Lagrange reconstruction loses order and the code (correctly) falls back to the exact straight-chord Shoelace area, recovering the discrete-mesh volume but reintroducing the $O(h^2)$ geometric bias there. (iv) The whole construction exploits the block factorisation (3) of the extruded geometry; a fully three-dimensional curvilinear map would require a 3×3 Jacobian quadrature rather than the present area $\times$ chord separation, though the median-dual and corrector theory carry over unchanged. (v) The host-only verification harness of §7 reproduces the solver kernels (the GL-3 nodes/weights, the degree-5 Lagrange basis, the centred/clamped stencil selector, the tensor reconstruction, and the Shoelace area) as *hand-copied snapshots*, because the originals are CUDA/MPI-coupled and cannot be linked into a standalone program; the harness must therefore be kept in manual sync with `evolution.h` and will otherwise silently validate stale code—a known limitation that a shared header would remove, at the cost of editing the solver source.

Generality. Nothing in §3–§4 is specific to D3Q19 beyond the existence of a zero-velocity rest population to carry the momentum-silent shift; any lattice with $\mathbf{e}_0 = \mathbf{0}$ admits the identical corrector, and any body-fitted/finite-volume discretisation with a partition-of-unity nodal volume admits the identical volume-weighted mean.

11 Conclusion

We have formulated, analysed, verified, and documented a Jacobian-weighted volumetric density corrector for mass conservation in curvilinear lattice-Boltzmann flows. The theory establishes that the physical mass $M = \int_{\Omega} \rho dV$, and hence the volume-weighted mean $\langle \rho \rangle_V$, is the only mesh-invariant target, and proves that a uniform shift into the rest population f_0 restores $\langle \rho \rangle_V = 1$ in a single, momentum-preserving step. The median-dual control volumes satisfy an exact partition-of-unity identity, and the cell areas that build them are integrated either exactly (straight-chord Shoelace) or to sixth order (Jacobian 3×3 Gauss–Legendre of a degree-5 Lagrange reconstruction of the in-plane metric). An independent host-only harness confirms GL-3 degree-5 exactness, sixth-order convergence of the Jacobian method against a textbook $O(h^2)$ for the chorded method, and algebraically exact, momentum-preserving mass restoration to the reduction-rounding floor ($\sim 10^{-16}$ in the small verification harness, $\sim 10^{-12}$ at production scale). A complete theory $\leftrightarrow$ code map ties every symbol to the D3Q19 GILBM source, and a machine-checked record establishes that the MRT collision and Guo forcing inject no spurious mass—leaving the volumetric corrector as the sole, controlled mass channel. The result is a mass governor that is mesh-invariant by construction, exact in one step in exact arithmetic, and limited in practice only by the well-conditioned, tree-summed parallel reduction.

A MRT collision verification record

The following is the verbatim machine-checked verification record for the MRT collision operator and Guo forcing used by the solver (mixed English/Traditional-Chinese audit log, reproduced as run).

```
=== MRT/LBM Collision 啟動驗證 ===  
結論：正確
```

Conserved Moment Indices (由 M 矩陣數值證明，非靠變數名/註解):

```
k=0 density (s=0, M row = {1,1,1,...,1} 全為 1)  
k=3 rho*u_x (s=0, M row == e_x = {0,1,-1,0,0,0,0,1,-1,1,-1,1,-1,1,-1,0,0,0,0})  
k=5 rho*u_y (s=0, M row == e_y = {0,0,0,1,-1,0,0,1,1,-1,-1,0,0,0,0,1,-1,1,-1}) <- streamwise  
k=7 rho*u_z (s=0, M row == e_z = {0,0,0,0,0,1,-1,0,0,0,0,1,1,-1,-1,1,1,-1,-1})
```

檢查結果:

```
[v] Relaxation rates: conserved s[0]=s[3]=s[5]=s[7]=0.0  
[v] Collision: conserved moments 不被 relax (projection f_out=f_B-K(f-feq))  
[v] K 矩陣: max|M_conserved.K| = 1.66e-15 < 1e-12 (k=0,3,5,7 四列全部 ~0)  
[v] Forcing: density 矩=0 (sum_i F0=0), y-momentum 矩=1, (1-s_k/2)=1 因 s_k=0  
[v] Force 未被覆蓋: f_out[a]=f_B[a]-relax 之後 += dt*Force0*(Fproj+u*Fu+v*Fv+w*Fw)  
[v] 註解正確性: u/v/w = spanwise/streamwise/wall-normal 標註正確
```

FORCE_HERMITE_ORDER=2 驗證:

```
二階完整 Guo: F_i = w_q*Force*[3(c_y-v) + 9(c.u)c_y]  
sum_i F_i = 1.4e-17 ~ 0 (質量守恒), sum_i F_i*c_y = 1.0 (y-動量守恒)  
Host 拆成 4 基底 F0=w*3c_y, Fu=w*9c_x c_y, Fv=w*(9c_y^2-3), Fw=w*9c_z c_y  
kernel 線性重組 Fproj + u*Fu + v*Fv + w*Fw 等價於完整二階 (容差 1e-12)  
一階若啓用為 w_q*3*c_y*Force (無 -v 項)
```

相關檔案:

```
MRT matrix: MRT_Matrix.h:26 (M), :48 (Mi), :4 (Relaxation s)  
Relaxation: mrt_projection_host.h:20, variables.h:118 (USE_MRT), :331 (FORCE_HERMITE_ORDER=2)  
Collision: gilbm/evolution_gilbm/0.collision.h:38-78, :65 (K), :69-73 (Guo force)  
Forcing build: mrt_projection_host.h:127-170 (K+Fproj), :103-125, :264-416 (Verify)  
Constants: gilbm/evolution_gilbm/0.shared_code.h:48-52 (__constant__ K, Fproj*)
```

Bug (若有): 無。質量與動量守恒矩、conserved s=0、K 投影零化、force 注入與半力係數、二階 Guo 拆解全部通過 1e-12 容差。

驗證細節: 以 numpy 重建 M、解析 Mi (Mi.M 恆等誤差 1.35e-15)、s=diag(0,1.19,...), 組 K=Mi.S.M。M[k].K 對 k in {0,3,5,7} 最大 1.66e-15。二階直接 Guo 力 sum=1.4e-17、sum.c_y=1, 確認 split 基底重組無誤。

References

- [1] Y. H. Qian, D. d'Humières, and P. Lallemand, Lattice BGK models for Navier–Stokes equation, *Europhysics Letters* **17**(6), 479–484, 1992.
- [2] D. d'Humières, I. Ginzburg, M. Krafczyk, P. Lallemand, and L.-S. Luo, Multiple-relaxation-time lattice Boltzmann models in three dimensions, *Philosophical Transactions of the Royal Society A* **360**(1792), 360–451, 2002.

- [3] Z. Guo, C. Zheng, and B. Shi, Discrete lattice effects on the forcing term in the lattice Boltzmann method, *Physical Review E* **65**(4), 046308, 2002.
- [4] T. Krüger, H. Kusumaatmaja, A. Kuzmin, O. Shardt, G. Silva, and E. M. Viggien, *The Lattice Boltzmann Method: Principles and Practice*, Springer, Cham, 2017.
- [5] X. He and L.-S. Luo, Theory of the lattice Boltzmann method: From the Boltzmann equation to the lattice Boltzmann equation, *Physical Review E* **56**(6), 6811–6817, 1997.
- [6] O. Filippova and D. Hänel, Grid refinement for lattice-BGK models, *Journal of Computational Physics* **147**(1), 219–228, 1998.
- [7] J. Stoer and R. Bulirsch, *Introduction to Numerical Analysis*, 3rd ed., Springer, New York, 2002.
- [8] P. J. Davis and P. Rabinowitz, *Methods of Numerical Integration*, 2nd ed., Academic Press, Orlando, 1984.
- [9] T. J. Barth, Aspects of unstructured grids and finite-volume solvers for the Euler and Navier–Stokes equations, *AGARD/VKI Lecture Series* 1991-05 (von Karman Institute), 1991.
- [10] J. Fröhlich, C. P. Mellen, W. Rodi, L. Temmerman, and M. A. Leschziner, Highly resolved large-eddy simulation of separated flow in a channel with streamwise periodic constrictions, *Journal of Fluid Mechanics* **526**, 19–66, 2005.
- [11] M. Breuer, N. Peller, C. Rapp, and M. Manhart, Flow over periodic hills—Numerical and experimental study in a wide range of Reynolds numbers, *Computers & Fluids* **38**(2), 433–457, 2009.
- [12] S. Succi, *The Lattice Boltzmann Equation for Fluid Dynamics and Beyond*, Oxford University Press, Oxford, 2001.